

289.60 Kcal

A few structural things stood out that matter more than any single snippet:

Scraping a logged-in web UI is your single largest fragility source. The DOM-parsing approach (concern #2) is not a bug to be patched — it's the root cause of most of your instability. Coinglass renders indicator values into TradingView iframe legends that only update on *mouse hover*, which is why your seeding logic has that elaborate arrow-key-and-mouse-move dance. That is inherently brittle and will break on any UI change. I'll come back to this, because the right answer to #2 substantially simplifies #1, #3, and #4 as well.

Your TradeTracker._save_history is buggy and will throw silently. `self.history` is a list (see `_load_history` which does `json.load` into a list and iterates for `t in self.history`), but `_save_history` treats it as a dict of dicts:

```
json.dump({sym: {k: list(v) for k, v in sym_hist.items()} for sym, sym_hist in
self.history.items()}, f, ...)
```

A list has no `.items()`, so every save raises `AttributeError`, which your bare `except:` swallows. Your trade history is silently never persisted. Fix:

```
def _save_history(self):
    try:
        tmp = self.log_file + ".tmp"
        with open(tmp, "w") as f:
            json.dump(self.history, f, indent=2, default=str)
        os.replace(tmp, self.log_file)
    except Exception as e:
        print(f"[TradeTracker] save error: {e}", file=sys.stderr)
```

Also note `_load_history` double-counts: it sets `self.current_capital += pnl` inside the loop but `current_capital` was already initialized to `initial_capital`, and separately maintains `temp_equity` — that's fine, but if `json.load` returns a dict on some malformed file it'll silently `except: pass` and leave you with a corrupt state. Never use bare `except:` for persistence logic.

Bare except: clauses are pervasive (`_load_history`, `parse_value`, `close`, dozens in the scraper). Each one is a place where a real bug hides as "it just returns N/A." At minimum catch `Exception` and `log`. This is the difference between "the engine is working" and "the engine is silently degrading and you can't tell."

self.strategies is referenced before it may exist in `_process_signal_locked` you do for `strat_obj in getattr(self, 'strategies', [])` — defensive, good — but you rely on `main()` setting `trade_tracker.strategies = strategies` after construction. If any code path calls `process_signal` before that assignment, ADX/ATR lookups silently return defaults. Pass strategies into the constructor instead of monkey-patching.

Now to your four concerns.

Concern 1: End-to-end exception handling and self-healing

Your instinct is right, but the current design has a specific weakness: **you have many independent daemon threads** (`tv_scraper_thread`, `footprint_scraper_thread`, `strategy_eval_worker`, OBD WS, anomaly agent), and if any of them dies, **nothing restarts it**. The `tv_scraper_thread` has an inner reconnect loop, which is good, but `strategy_eval_worker` and `footprint_scraper_thread` will exit permanently on an unhandled exception outside their inner try. A silently-dead strategy thread means the engine "runs" but never trades — the worst failure mode because it looks healthy.

The robust pattern is a **supervised thread wrapper** that restarts any worker with exponential backoff, plus a heartbeat so a hung (not crashed) thread is also detected. Here's a drop-in supervisor:

```
import threading, time, traceback

class SupervisedWorker:
    """Restarts a target callable forever with capped exponential backoff.
    Emits a heartbeat timestamp the watchdog can poll to detect hangs."""

    def __init__(self, name, target, on_fatal=None,
                 base_backoff=2.0, max_backoff=60.0, log_print):
        self.name = name
        self.target = target # callable(heartbeat_cb) -> runs its own loop
        self.on_fatal = on_fatal
        self.base_backoff = base_backoff
        self.max_backoff = max_backoff
        self.log = log
        self.last_beat = time.time()
        self._stop = threading.Event()
        self._thread = threading.Thread(target=self._run, name=name, daemon=True)

    def _beat(self):
        self.last_beat = time.time()

    def start(self):
        self._thread.start()

    def stop(self):
        self._stop.set()

    def _run(self):
        backoff = self.base_backoff
        while not self._stop.is_set():
            try:
                self._beat()
                # target should call self._beat periodically inside its own loop
                self.target(self._beat, self._stop)
                # Clean return means the worker finished intentionally.
                self.log(f"[{self.name}] worker returned cleanly; not restarting.")
                return
            except Exception as e:
                self.log(f"[{self.name}] crashed: {e}\n(traceback.format_exc())")
                if self.on_fatal:
                    try: self.on_fatal(e)
                    except Exception: pass
                if self._stop.wait(backoff):
                    break
                backoff = min(backoff * 2, self.max_backoff)
```

Refactor each worker to accept `(beat, stop)`:

```
def _tv_scrape_worker(beat, stop):
    with sync_playwright() as pw:
        browser = pw.chromium.connect_over_cdp(CDP_BASE, no_defaults=True)
        ctx = browser.contexts[0]
```

```
scraper = CoinglassScraper()
scraper.context = ctx
consecutive_errors = 0
try:
    while not stop.is_set():
        data = scraper.get_live_data()
        if data:
            feed.update_from_layout(data)
            consecutive_errors = 0
        else:
            consecutive_errors += 1
            if consecutive_errors > 5:
                raise RuntimeError("TV scraper: >5 empty scrapes, forcing reconnect")
            beat()
            stop.wait(2.0)
finally:
    try: browser.close()
    except Exception: pass
```

Then a single watchdog that detects *hung* threads (crashed ones self-restart via the supervisor):

```
def watchdog(workers, stall_secs=120, log=print):
    while True:
        now = time.time()
        for w in workers:
            if now - w.last_beat > stall_secs:
                log(f"[Watchdog] {w.name} stalled")
                log(f"({now - w.last_beat:.0f}s). Signalling restart.")
                # Simplest robust action: stall supervisor loop restarts on next tick.
                # For a truly hung thread you generally must let the process restart -
                # Python can't safely kill threads. See note below.
            time.sleep(15)
```

Important caveat on the last point: Python cannot forcibly kill a hung thread. If a Playwright page.evaluate call blocks indefinitely (which happens when a tab is discarded or Chrome hangs), the supervisor can't rescue it — the thread is stuck inside a C call. The only truly robust recovery is a **process-level supervisor**: run Engine_3 under something that restarts the whole process (systemd on Linux, NSSM or a Task Scheduler wrapper on Windows, or a thin parent-process launcher). Given you're on Windows (msvcrt, taskkill), I'd wrap the whole thing:

```
# launcher.py - run this instead of Engine_3 directly
import subprocess, sys, time
while True:
    p = subprocess.run([sys.executable, "Engine_3.py"] + sys.argv[1:])
    print(f"[Launcher] Engine exited with {p.returncode}. Restart in 10s.")
    time.sleep(10)
```

Combine per-thread supervision (fast recovery from transient errors) with process-level restart (recovery from true hangs). For the Cloudflare/captcha case specifically: detect if (page.title() containing "Just a moment", or a #challenge-form element) and treat it as a fatal error that triggers a longer backoff, since hammering it makes it worse.

Wrap main() itself so it never crashes the process before the launcher sees it:

```
def main():
    try:
        _main_impl()
    except KeyboardInterrupt:
        print("[Exit] Clean shutdown.")
    except Exception as e:
        print(f"[FATAL] main() crashed: {e}\n{traceback.format_exc()}", file=sys.stderr)
        sys.exit(1) # non-zero so the launcher restarts
```

Concern 2: DOM selector brittleness — intercept the network instead

This is the highest-leverage change you can make, and you named the right answer yourself: **intercept the network traffic instead of parsing the DOM**. The Coinglass TradingView charts fetch their indicator data over HTTP/WebSocket before rendering it. If you capture that upstream, you get clean numeric JSON, you eliminate the hover-to-update problem entirely, you don't care what CSS classes Coinglass uses, and it's dramatically faster.

Playwright can passively observe every response the page makes:

```
def attach_network_capture(page, store, store_lock, log=print):
    """Passively snoop chart data responses. Adjust url_match to the real
    Coinglass API paths you observe in DevTools > Network."""
    def looks_like_chart_data(url):
        return any(k in url for k in (
            "/api/", "openInterest", "fundingRate",
            "longShort", "liquidation", "cvd", "kline",
        ))
    def on_response(resp):
        try:
            url = resp.url
            if not looks_like_chart_data(url):
                return
            ct = resp.headers.get("content-type", "")
            if "application/json" not in ct:
                return
            payload = resp.json() # already-parsed numeric data
            with store_lock:
                store.setdefault("_raw", []).append((time.time(), url, payload))
        except Exception as e:
            log(f"[NetCapture] parse skip {url}: {e}")
    page.on("response", on_response)
```

For the WebSocket path (TradingView streams updates over WS), Playwright exposes frames directly:

```
def attach_ws_capture(page, on_message, log=print):
    def on_ws(ws):
        log(f"[WS] opened {ws.url}")
        ws.on("framereceived", lambda payload: on_message(ws.url, payload))
    page.on("websocket", on_ws)
```

The one-time cost: open Chrome DevTools, watch the Network tab while a chart loads and updates, and

Identify the exact request URLs and response shapes. Map those to your symbol → metric structure once. After that, EXTRACTION_JS and the entire seeding hover-dance can be retired.

Two honest caveats. First, network responses can be batched, paginated, or delivered as opaque binary WS frames — Coinglass may not make this as clean as the DOM. Test before committing. Second, and more importantly, **the cleanest solution is to stop scraping the UI altogether**. You're already using Coinglass; they sell a proper REST API. And several of your metrics (OI, funding, L/S ratio, klines/CVD, liquidations) are available directly from Binance's public futures API, which you *already* call in FootprintScraper and get_raw_klines. Every metric you can source from a documented API is one fewer thing that breaks when a designer moves a <div>. I'd rank your options: official Coinglass API > Binance public API for what it covers > network interception > DOM scraping. Treat DOM scraping as the fallback of last resort, and keep it behind a resilient proxy (regex on innerText rather than class names) so a UI tweak degrades rather than crashes:

```
# More resilient than class-name traversal: match on label text + number nearby.
NUM = r'[-~?](\d,)+\.\d*[KMB]?%'
PATTERNS = {
    "funding": re.compile(r'Funding Rate[s]?D*({NUM})', re.I),
    "oi": re.compile(r'Open Interest\D*({NUM})', re.I),
    "ls_ratio": re.compile(r'Long/Short Ratio\D*({NUM})', re.I),
}
def parse_from_text(full_text):
    out = {}
    for k, pat in PATTERNS.items():
        m = pat.search(full_text)
        out[k] = m.group(1) if m else "N/A"
    return out
```

Concern 3: Thread safety and memory leaks over 24/7 runtime

You have real risks here, some already handled and some not.

Handled well: UnifiedDataFeed.history uses deque(maxlen=14410), so per-key history is bounded. add_to_history's slice-prune is a bit redundant given the maxlen deque (the del hist[: -max_points] won't do much on a bounded deque, and max_points ≈ 14400 matches the maxlen anyway), but it's not leaking.

Not handled — this is your actual leak risk: MLSignalFilter.models_cache and

BaseTrendStrategy._raw_klines_cache grow without bound. The klines cache keys on f* {symbol}_{interval_mins}_{limit}* so it's bounded by distinct combinations — fine. But models_cache keys on strategy label; bounded by strategy count — also fine. The genuinely unbounded one is anything keyed on time or trade IDs. Audit for dicts that accumulate per-event keys. Your active_anomalies and history in AnomalyAgent are pruned — good.

Playwright object accumulation is the subtler risk. Every page.evaluate returns a JS handle that Playwright bridges to Python. These are garbage-collected, but over millions of calls in a 24/7 loop, and especially if you ever create pages/contexts inside the loop, you can leak. Your tv_scraper_thread correctly creates one sync_playwright() + browser per reconnect and reuses it — good. **Never create a page/context per iteration.** One thing to add: periodically recycle the whole Playwright connection (e.g., every few hours) even without an error, because CDP connections accumulate cruft over very long lifetimes.

```
def tv_scrape_worker(beat, stop):
    RECYCLE_SECS = 3 * 3600
    while not stop.is_set():
        started = time.time()
        with sync_playwright() as pw:
            browser = pw.chromium.connect_over_cdp(CDP_BASE, no_defaults=True)
            try:
                scraper = CoinglassScraper(); scraper.context = browser.contexts[0]
                while not stop.is_set() and time.time() - started < RECYCLE_SECS:
                    data = scraper.get_live_data()
                    if data: feed.update_from_layout(data)
                    beat(); stop.wait(2.0)
            finally:
                try: browser.close()
                except Exception: pass
```

Genuine thread-safety bug: TradeTracker._update_live_pnl_locked iterates self.active_trades and, inside the loop, calls self.process_signal(...) for closes — but process_signal acquires self.lock (an RLock, so re-entry is fine) and mutates self.active_trades via .pop. You defend against this by collecting into to_close first and closing after the iteration, which is correct. Good. But get_render_snapshot reads self.history[-3:] under lock while process_signal appends under lock — consistent. The one place to double-check: AnomalyAgent writes feed.latest_data[symbol] ['anomaly_flag'] under feed.lock, while the render loop reads via get_latest() (also under lock) — consistent.

Recommendation: add explicit memory instrumentation so you're not guessing. Log RSS and key cache sizes every few minutes:

```
import psutil, os
def log_health(feed, tracker):
    p = psutil.Process(os.getpid())
    print(f'[Health] RSS={p.memory_info().rss/1e6:.0f}MB "
          f"threads={threading.active_count()} "
          f"active_trades={len(tracker.active_trades)}")
```

If RSS climbs monotonically over 24h, you have a leak; if it plateaus, you don't. Measure rather than speculate.

Concern 4: Performance — single JSON payload per tab

Your EXTRACTION_JS already does most of this correctly: it loops all 9 windows and returns one result object in a single page.evaluate round trip per tab. So you're at **2 evaluations per scrape cycle**, not 18 — that's the right structure. The inefficiency is inside the JS: for each legend you re-run querySelectorAll('[class*="legend-"]') and string.split every legend's innerText, which is wasteful but cheap relative to the CDP round-trip latency (typically 5–50ms each).

The real performance win isn't micro-optimizing the JS — it's **eliminating the round trip's dependency on hover/render**. If you move to network interception (concern #2), the data arrives pushed, and your "scrape" becomes a lock-guarded dict read with zero cross-process cost. That's orders of magnitude faster than any page.evaluate tuning.

If you stay on DOM extraction, two concrete improvements. First, cache the iframe document references

and query once, returning a flat typed structure so Python does no string parsing.

```
() -> {  
  const out = {};  
  for (let i = 1; i <= 9; i++) {  
    const c = document.getElementById(`tv_chart_container_win${i}`)  
    || (i === 1 ? document.getElementById('tv_chart_container_main') : null);  
    const iframe = c && c.querySelector('iframe');  
    if (!iframe) { out[`win${i}`] = null; continue; }  
    let doc; try { doc = iframe.contentDocument || iframe.contentWindow.document; }  
    catch(e) { out[`win${i}`] = {err: 'cross-origin'}; continue; }  
    const legends = doc.querySelectorAll('[class*="legend-"]');  
    const rec = {};  
    for (const el of legends) {  
      const t = (el.innerText||'').replace(/-/g, '-');  
      // return raw label+value pairs; let Python map them once  
      const last = t.split("\n").map(s=>s.trim()).filter(Boolean).pop();  
      if (t.includes('Open Interest')) rec.oi = last;  
      else if (/Funding Rate/i.test(t)) rec.funding = last;  
      else if (/Long\Short/i.test(t)) rec.ls = last;  
      // ...etc  
    }  
    out[`win${i}`] = rec;  
  }  
  return out; // one structured payload, parsed by Playwright as JSON  
}
```

Second, don't `page.evaluate` more often than the data actually changes. Your 15m-timeframe strategy does not need sub-second scraping — the underlying candles update every 15 minutes and the live legend updates maybe every few seconds. Your `stop.wait(2.0)` cadence is already reasonable; I wouldn't push it faster. Scraping faster than the data changes just burns CDP bandwidth and increases your memory-churn surface for no signal benefit.

Summary of priorities

If I were ranking these by return on effort: first, rotate that leaked password and move secrets to env vars — non-negotiable and takes minutes. Second, migrate off DOM scraping to the Coinglass/Binance APIs or network interception; this single change dissolves most of concerns 1–4. Third, add the process-level launcher plus per-thread supervision, since Python can't kill hung threads and Playwright hangs are your likeliest 3am failure. Fourth, fix the `_save_history` dict/list bug and replace bare `except:` clauses with logged `except Exception`, because right now the engine can degrade silently and you'd never know. The memory concern (#3) is real but secondary — instrument it with `psutil` and let 24 hours of data tell you whether it's actually leaking rather than pre-optimizing.

One closing note on the trading logic itself, since you asked for an end-to-end audit: I reviewed this purely as software architecture, not as a trading strategy. Whether the Zeno sizing, the prop-firm drawdown guards, or the meta-labeler thresholds are *financially* sound is outside what I can assess from the code, and you shouldn't take this review as any endorsement that the system will be profitable or safe with real capital. Test extensively on paper first.

Want me to go deeper on any one of these — for example, sketch the full network-interception mapping workflow, or write out the complete supervised-worker refactor of `main()`?

📄 📌 💬 ...

AI Chat • The AI Costco — one membership →

Ask anything, create anything

+ Claude Opus 4.8 ▾

🗣️ ↩️